SAGE: Self-Assessed Gradient-Based Enhancement for Test-Time Alignment of Large Language Models

Anonymous submission

Abstract

We present SAGE, a test-time alignment method for refining large language model (LLM) outputs at inference time without retraining or relying on external reward models. SAGE uses the same LLM as both generator and self-judge: candidate responses are scored using a contrastive label judgment prompt and the resulting log-probability margin serves as a continuous preference signal. An iterative refinement algorithm then uses these scores to form good and bad candidate sets, extract a textual gradient, and guide subsequent hypothesis generations under a token budget. Across MATH-500, AlpacaEval, and XSTest, with Qwen3-8B and Llama-3.1-8B-Instruct, SAGE consistently improves accuracy and preference metrics relative to existing test-time alignment approaches, with ablations showing that these gains are robust under fixed inference budgets and do not rely on external reward models. The approach is resource-lean and plug-and-play, and we release the code and prompts to facilitate adoption.

Code —
https://anonymous.4open.science/r/SAGE2026-8309

1 Introduction

Large language models (LLMs) pre-trained on vast amounts of unlabeled text have demonstrated strong generalization across a wide range of downstream tasks. Nonetheless, real deployments often require adapting behavior to new domains, user preferences, or safety constraints without the time, data access, or compute needed for retraining. Test-time alignment addresses this need by improving outputs **during inference**, while keeping model weights fixed. It typically allocates a small amount of additional inference compute and elicits interpretable preference signals to steer generation toward more helpful responses aligned with user preferences (Li et al. 2025). This paradigm offers a practical path to domain adaptation and preference alignment under privacy, compliance, and latency constraints, without modifying the pretrained model.

Despite recent progress, existing state-of-the-art test-time alignment approaches, such as TPO (Li et al. 2025) and ARGS (Khanov, Burapacheep, and Li 2024), still face several practical limitations. First, many of these methods rely on a separate reward model for scoring, which adds memory and latency overhead and introduces an additional component that can itself be misaligned or exploited. Second, relying solely on the single highest- and lowest-scoring responses for textual critiques can lead to noisy and outlier-sensitive feedback that fails to capture strengths or failure patterns across candidates, resulting in unstable or suboptimal refinement.

We propose SAGE, which treats the base LLM as both a generator and a soft self-judge. Instead of relying on an external reward model, SAGE elicits a contrastive log-probability label margin from a compact judging prompt, which enables fine-grained, model-internal preference scoring. A key component is an automatic contrastive grouping mechanism that forms representative good and bad sets, yielding a more stable textual 'gradient' than extreme-only (best vs. worst) critiques and reducing sensitivity to outliers. The iterative loop then conditions generation on this gradient to refine future samples. In effect, SAGE leverages the LLM to surface its own implicit reward signal for test-time alignment, avoiding the need for additional reward models while operating under a small inference budget. An overview of SAGE in comparison to common test-time alignment variants is presented in Figure 1.

We conduct experiments on MATH-500 (Hendrycks et al. 2021), AlpacaEval (including length-controlled LC win rate) (Dubois et al. 2024), and XSTest (Röttger et al. 2024), using Qwen3-8B (Yang et al. 2025) and Llama-3.1-8B-Instruct (Grattafiori et al. 2024).[1] These benchmarks cover math reasoning (MATH-500), general instruction following (AlpacaEval), and safety/correctness stress tests (XSTest). We report accuracy on MATH-500/XSTest and win rate and length controlled (LC) win rate for AlpacaEval, comparing against commonly used inference-time baselines, including Best-of-N, TPO (Li et al. 2025) which relies on external reward model, and SPO (Xiang et al. 2025). SAGE consistently improves performance across these benchmarks without relying on an external reward model. On Qwen3-8B, SAGE

[1] For Llama we use the W8A16 weight-quantized release (`Meta-Llama-3.1-8B-Instruct-quantized.w8a16`) to fit the serving budget; its absolute scores are accordingly lower than fp16 reference numbers.

improves accuracy and preference metrics across all evaluated domains. AlpacaEval win rate vs. GPT-4 increases from 57.8 to 74.9, XSTest accuracy from 89.3 to 93.5, and MATH-500 accuracy from 84.4 to 92.0. SAGE also outperforms inference-time baselines that rely on explicit reward models (e.g., Best-of-N+RM, TPO) and generic LLM-as-a-judge approaches, under comparable budgets. The same procedure also applies to Llama-3.1-8B-Instruct, suggesting that SAGE's self-judged textual-gradient refinement generalizes across model families.

We summarize our contributions as follows:

- We introduce SAGE, a test-time alignment method in which the base LLM provides its own soft reward model via a contrastive label margin. As a result, the method avoids external reward models, reducing system complexity and potential reward signal misalignment.
- We design a grouped selection and textual-gradient mechanism that constructs good and bad sets and distills concise guidance from the judge. Compared to best–vs.–worst ranking-based feedback, this leads to more stable refinement by reducing sensitivity to outliers and improving transfer across iterations.
- We provide a practical self-judging recipe, called multi-aspect tagging with weights and confidence-threshold calibration, that turns discrete tags into a continuous score. This scoring scheme improves hypothesis ranking and provides a reliable correctness signal, particularly for verifiable tasks.
- Finally, we present an iterative test-time alignment procedure that integrates self-judged scoring, grouped selection, and textual gradients to progressively improve generations. Code, prompts, and scoring configurations are released to support reproducibility at: https://anonymous.4open.science/r/SAGE2026-8309.

2 Related Works

2.1 Preference alignment

Aligning pretrained LLMs with human preferences is often approached by updating model parameters using preference supervision. Existing methods span point-wise optimization, such as policy-gradient variants like PPO (Schulman et al. 2017), ReMax (Li et al. 2024), and KTO (Ethayarajh et al. 2024), which adjust the model using scalar feedback on individual outputs, as well as pairwise optimization approaches that leverage relative judgments. A central strand is DPO (Rafailov et al. 2023), which reformulates RLHF objectives to directly optimize the policy from pairwise comparisons. Subsequent work addresses overfitting and reduces reliance on reference policies, for example by constraining score gaps (Azar et al. 2024), or by removing or adapting the reference model (Meng, Xia, and Chen 2024; Kim et al. 2025; Gorbatovski et al. 2025). These training-time procedures are effective, but they require curated preference data and repeated fine-tuning, which can be slow to iterate and costly to deploy as requirements change.

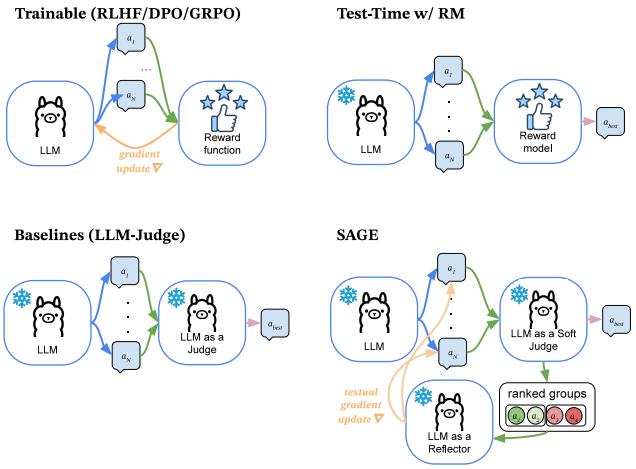


Figure 1: Comparison between test-time alignment methods. SAGE acts post-training and uses the base LLM as generator, soft self-judge with a calibrated contrastive score, and textual-gradient extractor unlike LLM-as-judge heuristics or external RM–based scoring.

2.2 Test-time alignment

Test-time alignment covers decoding heuristics, compute-allocation strategies, and judge-driven selection used at inference without changing model weights. Classical decoding includes greedy and beam search (Reddy 1976), which track multiple partial hypotheses. Although effective on structured tasks, these methods can over-concentrate probability mass and amplify local errors. Sampling-based decoding encourages diversity. For example, nucleus sampling (Holtzman et al. 2020) truncates low-mass tails, and contrastive search (Su and Collier 2023) penalizes degeneracy. However, both remain sensitive to hyperparameters.

Beyond fixed heuristics, adaptive search variants add flexibility, for example through alternative normalization schemes or affect-guided steering (Freitag and Al-Onaizan 2017; Huang et al. 2023), but they still rely on hand-tuned policies. A strong and widely used baseline is best-of-n (BoN): draw n candidates and pick the highest under a proxy. When paired with a reward model (RM), BoN+RM can deliver sizable gains via reranking (e.g., human-feedback pipelines such as Stiennon et al. 2020). However, it incurs additional RM training and maintenance costs and risks over-optimization when the proxy is misaligned. Replacing the RM with an LLM judge trades training for prompt engineering. In this setting, judge prompts can be used to verify intermediate steps (e.g., for math (Mahdavi et al. 2025)) or elicit preferences. Test-time preference optimization frameworks (Li et al. 2025) iterate textual feedback to improve candidates. These approaches often rely on discrete labels or scalarized heuristics, which can be coarse, sensitive to verbosity, and inefficient in their use of tokens.

Recent work scales test-time compute by allocating budget across perception, planning, and tool use (Zhu et al. 2025) or by guiding sampling with intermediate checkpoints (Wang

et al. 2025), at the cost of controller complexity and extra hyperparameters. TreeBoN (Qiu et al. 2025) speeds up BoN with speculative tree search driven by token-level preference signals.

Prompt-optimization methods aim to improve elicitation rather than search, for example through self-supervised prompt tuning (Xiang et al. 2025). However, these approaches often depend on seed exemplars and can struggle to transfer across domains.

Same-model refinement is the setting closest to ours: Self-Refine (Madaan et al. 2023) and Reflexion (Shinn et al. 2023) let one LLM draft, critique, and revise its own outputs through free-form feedback. A complementary line of work cautions how far self-evaluation can be trusted: models discriminate their own outputs no better than they generate them (Jiang et al. 2024), judges favor familiar text (Wataoka, Takahashi, and Ri 2024), a shared generator-evaluator invites reward hacking (Pan et al. 2024), and verbalized confidence is imperfectly calibrated (Tian et al. 2023). We therefore score candidates contrastively against explicit rubrics and validate the selection signal against gold answers rather than assuming its reliability (Section 4.4); under a matched generation budget on MATH-500 this structured signal stays about two points ahead of free-form self-critique.

In contrast, SAGE uses the base LLM as both generator and soft judge and replaces brittle discrete votes with a calibrated continuous margin computed directly from a judging prompt. The resulting score supports fine-grained ranking, and a concise textual "gradient" then steers subsequent sampling. This preserves BoN-style simplicity and avoids external reward models and heavy heuristics. The approach also fits strict token budgets through lightweight judging and caching.

3 Proposed Method

3.1 Notation and Problem setup

Let M denote a pretrained (frozen) language model. For a prompt $p \in \mathcal{P}$, the model M induces a conditional policy $\pi_M(h \mid p)$ over responses $h \in \mathcal{X}$. At test time, alignment is typically driven by a scalar preference signal $s(p, h)$, for example from a reward model or an LLM judge, which scores how well h matches human preferences. Our test-time alignment goal is to select a response that maximizes this signal, without updating M:

$$h^\star(p) \;\in\; \arg\max_{h \in \mathcal{X}} s(p, h)\,, \tag{1}$$

Concretely, test-time alignment methods implement a procedure that interacts with M to propose, score, and refine candidates at inference time.

We identify two practical shortcomings of existing test-time alignment approaches: First, many methods rely on a separate reward model for scoring, which adds memory, latency, and maintenance overhead and can introduce proxy mismatch or reward hacking. Second, forming a stable textual 'gradient' for improving prior generations is difficult for methods that compare only the single best and worst candidates (e.g., TPO (Li et al. 2025)): such extreme-only feedback is outlier-sensitive and often produces noisy guidance that fails to capture broader error patterns across the candidate set. In the next section, we describe our solution to address these limitations.

3.2 SAGE

In this work, we propose SAGE, which uses a single LLM as the generator, the soft judge (reward model), and the textual-gradient estimator. The method assigns soft scores to rollouts and dynamically constructs contrastive groups to obtain a textual gradient, which is then used to further improve generation quality.

At a high level, SAGE uses the base LLM as both generator and soft judge. It draws small batches of candidates, scores them using a calibrated contrastive-label signal, aggregates evidence across groups to extract a concise textual 'gradient', and conditions subsequent sampling on this guidance, all under a fixed token budget. An overview of the method is shown in Figure 2.

Initial generation At the beginning of the algorithm, we generate an initial set of rollouts (hypotheses) using the LLM as the generator and store them in the rollout set $\mathcal{H}$, which is updated in subsequent iterations. These rollouts are then scored by the judge.

Judge

Structured judge prompts and label sets. We leverage the model's *implicit* preference signal at test time by asking the same LLM to judge its own outputs along a small set of human-interpretable aspects, such as usefulness, verification, completeness, and repetition. For each aspect $k \in \{1, \ldots, K\}$ we build a concise, deterministic judge prompt $P_j^{(k)}(p, h)$ that specifies an aspect-specific rubric and includes the original task prompt p together with the candidate response $h \in \mathcal{H}$, and requires the model to emit a decision inside dedicated tags, e.g. `<is_useful>yes|no</is_useful>` (see Appendix A). Parsing the tag yields a *hard* label $\hat{\ell}_k \in \Pi_k \cup \Sigma_k$, where Π_k (resp. Σ_k) is the set of positive (resp. negative) surface forms, e.g. $\{\texttt{yes}\}$ and $\{\texttt{no}\}$.

From tagged decisions to calibrated margins. Beyond the hard tag, we use the model's token-level probabilities to obtain *soft* confidence score. Let u_k be the fixed prefix in $P_j^{(k)}(p, h)$ up to, but not including, the first label token. For any $\ell \in \Pi_k \cup \Sigma_k$, we estimate the conditional log-probability $\log P(\ell \mid P_j^{(k)}, u_k)$ by summing the per-token log-probabilities for ℓ; if ℓ is multi-token, we evaluate the full string using teacher forcing and aggregate over its canonical tokenizations via log-sum-exp.[2] This yields a contrastive label margin for each aspect

$$\begin{aligned} s_k(h) \;=\; & \frac{1}{|\Pi_k|} \sum_{\ell \in \Pi_k} \log P\big(\ell \mid P_j^{(k)}, u_k\big) \\ & - \frac{1}{|\Sigma_k|} \sum_{\sigma \in \Sigma_k} \log P\big(\sigma \mid P_j^{(k)}, u_k\big), \end{aligned} \tag{2}$$

[2] With $\Pi = \{\text{"yes"}\}$, $\Sigma = \{\text{"no"}\}$, we aggregate over canonical variants of each word before forming the margin.

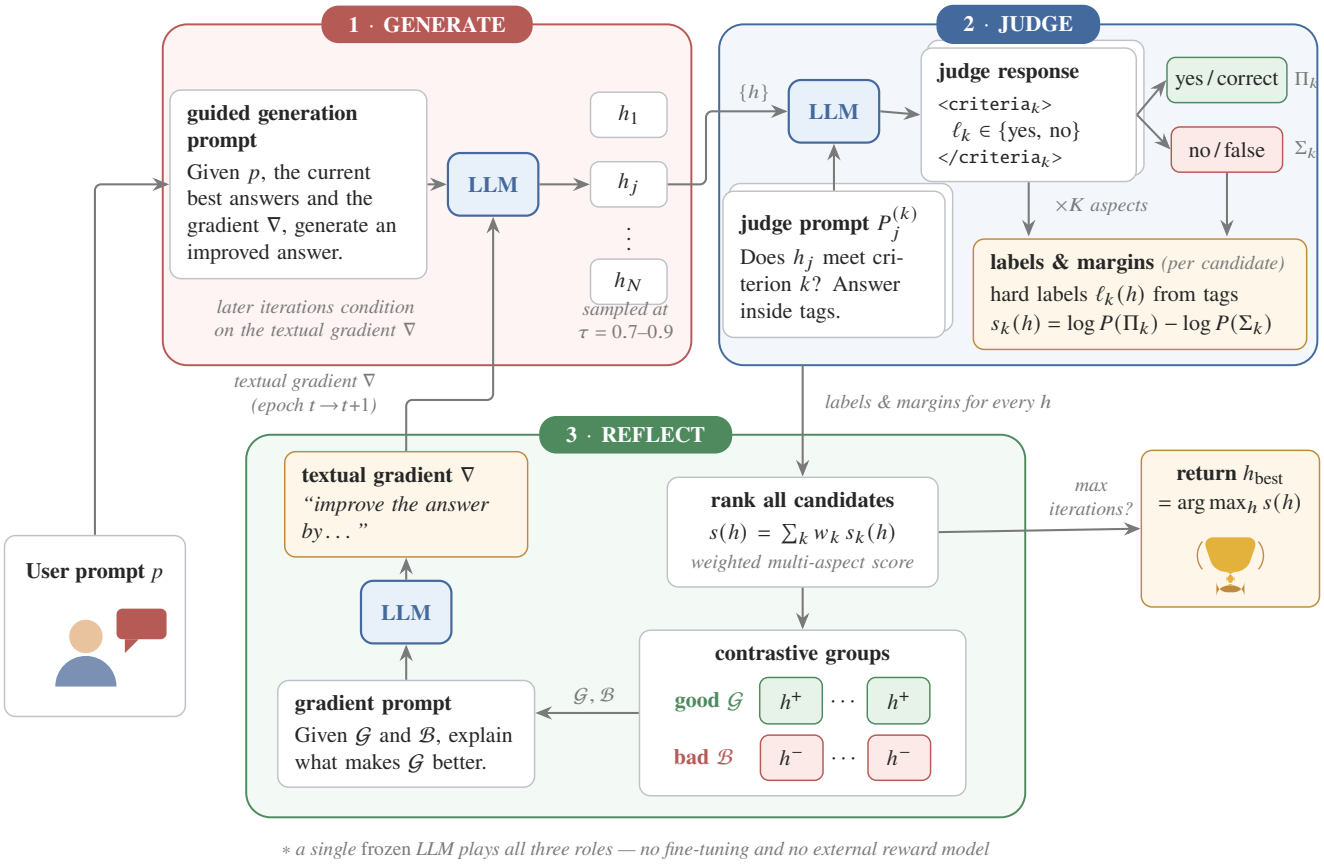


Figure 2: The SAGE framework: a single LLM acts as generator, self-judge, and reflector. Candidates are scored with contrastive label margins, partitioned into good and bad groups, and a textual gradient ∇ extracted from the groups guides the next generation round.

which quantifies how strongly the judge prefers positive over negative outcomes for a given aspect.

Multi-aspect aggregation. When multiple aspects are judged, we combine per-aspect margins with nonnegative weights w_k to form a scalar score

$$s_{\mathbf{w}}(h) \;=\; \sum_{k=1}^{K} w_k\, s_k(h), \tag{3}$$

used both to rank candidates and to form good and bad sets for the textual-gradient step. This construction makes the judge's outputs *explicit and auditable* via tags and it converts the model's own confidence over labels into a *calibrated* continuous preference signal for test-time alignment.

Calibration. We apply a temperature separation using a lower temperature for judging than for generation to reduce variance. We find the margin is robust to minor prompt rewording, and that its monotonicity correlates with preference on math-style tasks. Throughout, the margin serves as a ranking signal for selection and grouping; we do not rely on it as an absolute confidence estimate.

Reflection We form groups instead of comparing only extreme candidates. Contrasting a set of good responses $\mathcal{G}$ with a set of bad responses $\mathcal{B}$ yields a more stable and informative signal. By leveraging group-wise comparisons, we achieve more stable guidance and improved downstream transfer. Group formation proceeds in three steps. Let $\mathcal{H}$ denote the current pool of hypotheses and let $\hat{\ell}_k(h) \in \Pi_k \cup \Sigma_k$ denote the *hard* label emitted by the judge for aspect $k \in \{1, \ldots, K\}$ on candidate $h \in \mathcal{H}$. With the aggregate margin $s_{\mathbf{w}}(h)$ defined above, we construct $\mathcal{G}$ and $\mathcal{B}$ as follows (ties broken deterministically):

1. *Preliminary partition.* Define the preliminary good set $\mathcal{G}_{\text{pre}} = \{\, h \in \mathcal{H} : \forall k,\ \hat{\ell}_k(h) \in \Pi_k \,\}$, i.e., candidates that are positive on all aspects; and the preliminary bad set $\mathcal{B}_{\text{pre}} = \{\, h \in \mathcal{H} :\ \exists\, k \text{ s.t. } \hat{\ell}_k(h) \in \Sigma_k \,\}$, i.e., candidates with at least one negative aspect.

2. *Within-set ranking and truncation.* Let $m_{\min} \in \mathbb{N}$ be the per-group size cap: each side keeps at most $m_{\min}$ candidates (fewer when the corresponding preliminary set is smaller). Rank $\mathcal{G}_{\text{pre}}$ by decreasing $s_{\mathbf{w}}$ and $\mathcal{B}_{\text{pre}}$ by

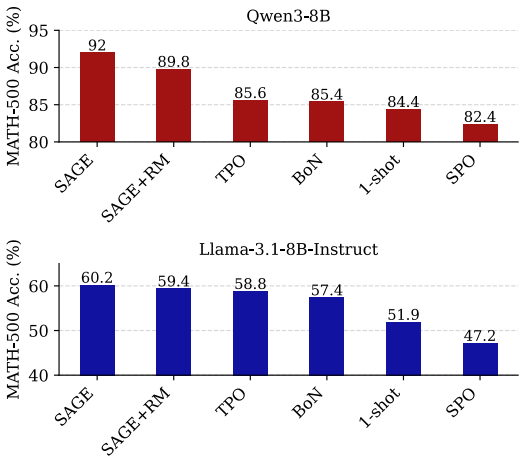


Figure 3: Results for SAGE on MATH-500 accuracy (%). SAGE outperforms strong inference-time baselines across different model families. The internal self-judge consistently yields higher final scores than external reward models on math reasoning (SAGE+RM).

increasing $s_{\mathbf{w}}$. Set

$$\mathcal{G} = \text{top } \min\big(|\mathcal{G}_{\text{pre}}|,\, m_{\min}\big) \text{ of } \mathcal{G}_{\text{pre}},$$
$$\mathcal{B} = \text{bottom } \min\big(|\mathcal{B}_{\text{pre}}|,\, m_{\min}\big) \text{ of } \mathcal{B}_{\text{pre}}.$$

3. *Fallback when one side is empty.* If $\mathcal{G}_{\text{pre}} = \varnothing$ (all candidates have at least one negative aspect), rank $\mathcal{H}$ by $s_{\mathbf{w}}$ and take $\mathcal{G} =$ top $m_{\min}$ of $\mathcal{H}$, $\mathcal{B} =$ bottom $m_{\min}$ of $\mathcal{H}$. Symmetrically, if $\mathcal{B}_{\text{pre}} = \varnothing$, set $\mathcal{B} =$ bottom $m_{\min}$ of $\mathcal{H}$ and $\mathcal{G} =$ top $m_{\min}$ of $\mathcal{H}$.

This procedure respects aspect-level hard decisions when available and uses the calibrated aggregate margin $s_{\mathbf{w}}$ to prioritize stronger evidence within each side. When the hypothesis pool $|\mathcal{H}|$ is sufficiently large, it yields nonempty sets $\mathcal{G}$ and $\mathcal{B}$, which stabilizes the subsequent textual-gradient step.

Each iteration thus scores the pool with cached margins, partitions it into good and bad sets, asks the LLM to summarize the changes that would turn bad responses into good ones (the textual "gradient"), and conditions the next $\min(N,\ B - \text{spent})$ samples on this guidance; we stop when B is exhausted or the epoch limit is reached and return the highest-scoring hypothesis (Algorithm 1).

Guided generation and scheduling The generation of subsequent hypotheses is conditioned on the original user prompt p, a concise form of the gradient g, and the k top-scoring exemplars from $\mathcal{G}$ under the composite score $s_{\mathbf{w}}(h)$. The decoding temperature is annealed over iterations, nucleus and top-k constraints are kept moderate, and maximum length is bounded to control judge cost. We also allow occasional "exploration" samples that ignore g with low probability to avoid premature convergence.

Algorithm 1 SAGE: Self-judged iterative response refinement over T epochs

Inputs: prompt p; number of epochs T; per-iteration batch size N; judge prompts $\{P_j^{(k)}\}_{k=1}^{K}$; aspect weights $\{w_k\}$; group-size cap $m_{\min}$
Initialize: hypothesis pool $\mathcal{H} \leftarrow \{\}$; cache $\mathcal{C} \leftarrow \{\}$
Step 0: Sample N hypotheses from M conditioned on p; add to $\mathcal{H}$
for $t = 1$ to T **do**
  For each $h \in \mathcal{H}$: obtain hard labels $\hat{\ell}_k(h) \in \{\Pi_k \cup \Sigma_k\}$ and per-aspect margins $s_k(h)$; compute $s_{\mathbf{w}}(h) = \sum_{k=1}^{K} w_k\, s_k(h)$; update $\mathcal{C}$
  Build preliminary sets $\mathcal{G}_{\text{pre}} = \{h \in \mathcal{H} : \forall k,\ \hat{\ell}_k(h) \in \Pi_k\}$, $\mathcal{B}_{\text{pre}} = \{h \in \mathcal{H} : \exists k,\ \hat{\ell}_k(h) \in \Sigma_k\}$
  Rank $\mathcal{G}_{\text{pre}}$ by decreasing $s_{\mathbf{w}}$ and $\mathcal{B}_{\text{pre}}$ by increasing $s_{\mathbf{w}}$; set $\mathcal{G} \leftarrow$ top $\min(|\mathcal{G}_{\text{pre}}|, m_{\min})$, $\mathcal{B} \leftarrow$ bottom $\min(|\mathcal{B}_{\text{pre}}|, m_{\min})$; if a side is empty, take top/bottom $m_{\min}$ from $\mathcal{H}$ by $s_{\mathbf{w}}$
  Query textual gradient $g \leftarrow$ LLM_improvements$(\mathcal{G}, \mathcal{B}, \{P_j^{(k)}\}, \{w_k\})$
  Condition on (p, g) and top exemplars from $\mathcal{G}$; sample N new hypotheses from M; add them to $\mathcal{H}$
end for
Return: $\arg\max_{h \in \mathcal{H}} s_{\mathbf{w}}(h)$

4 Experiments

We evaluate SAGE with Qwen3-8B (Yang et al. 2025) and Llama-3.1-8B-Instruct (Grattafiori et al. 2024) on MATH-500, AlpacaEval and XSTest. We report task metrics and preference-based wins, and study sensitivity to temperatures, the number of optimization epochs and the sampling sizes.

4.1 Benchmarks and setup

We compare against state-of-the-art test-time methods, including Test-time Preference Optimization (TPO) and Self-supervised Preference Optimization (SPO), as well as simpler baselines such as Best-of-N sampling and zero-shot generation.

4.2 Implementation details

We use Qwen3-8B and Llama-3.1-8B-Instruct as the base models. For zero-shot generations, we use nucleus sampling with temperature of 0.7 and top-p set to 0.95. For all evaluations that require a reward model, we use sfairXC/FsfairX-LLaMA3-RM-v0.1 (Dong et al. 2024).

For SPO, we perform prompt tuning using GPT-4.1 as both the judge and the prompt editor. To ensure fair measurement, with no prompt calibration on evaluation items, we calibrate on five out-of-domain examples per task from a similar domain. For MATH-500, we use AGIEval-Math (Zhong et al. 2024); for AlpacaEval we use a small subset of the Tulu instruction-following dataset (Lambert et al. 2025); and for XSTest we use Jailbreakbench (Chao et al. 2024). For TPO and Best-of-N, we use the official TPO repository (Li et al. 2025). Because this calibration is deliberately minimal and out-of-domain (a single prompt tuned from five examples, with no access to evaluation items), SPO's win rates should be read as a compute-matched operating point rather than its unconstrained ceiling: SPO is designed to exploit

a larger, in-domain prompt-optimization budget, which our inference-time-matched protocol does not grant it.

By default, we use temperature of 0.7 for generation and 0.1 for judging. For math problems, we typically use a higher generation temperature in early stages to encourage solution diversity. We ablate both generation and judging temperatures on MATH-500, as reported in Section 4.4. Prompts and evaluation criteria are provided in the Appendix. For math-oriented tasks, we anneal the generation temperature from 0.8–1.0 in the initial stage to 0.7 in subsequent stages. Unless otherwise noted, we sample $N = 7$ hypotheses per generation stage and run two optimization epochs per benchmark, excluding the initial generation stage. Because the good and bad sets are selected automatically, we do not fix their sizes; the median size of each group is two to three hypotheses. Unless stated otherwise, results are single runs with a fixed seed; selected analyses are repeated over three seeds. Regarding cost, two optimization epochs with $N = 7$ average roughly 71 seconds per MATH-500 problem on one H200 GPU, against about 14 seconds for Best-of-N reranked by the same judge; the overhead is dominated by the extra hypothesis generations, as judging reads a few forced label tokens and margins are cached. (H200 timings; the A100 wall-clock in Table 4 is correspondingly higher.)

All Qwen3-8B experiments in Table 1 use the non-thinking mode to keep the evaluation setup uniform across experiments. Nevertheless, SAGE is equally applicable in reasoning mode: results on AIME 2026 (Dekoninck et al. 2026) (Qwen3-8B and Qwen3.5-4B (thinking)), at additional model scales (Qwen3-1.7B, Qwen3-32B-FP8), and on further benchmarks (IFEval (Zhou et al. 2023), MMLU-Pro STEM (Wang et al. 2024)) are reported in Sections 4.5 and 4.6.

4.3 Main results

We report accuracy on MATH-500 and XSTest, and both pairwise win rate and length-controlled win rate on AlpacaEval.[3] Table 1 summarizes results for Qwen3-8B and Llama-3.1-8B-Instruct under SAGE alongside competitive baselines including TPO (Li et al. 2025), SPO (Xiang et al. 2025), Best-of-N, and the base models.

For Qwen3-8B, SAGE performs best on both math and general tasks. On MATH-500, it reaches the accuracy of 92.0, outperforming the base model as well as both Best-of-N and TPO. On AlpacaEval, it attains a win rate of 74.9 and LC win rate of 55.0, exceeding all baselines. This advantage on math tasks appears to be driven by self-judging. Qwen3-8B evaluates the correctness of its own generations more reliably than an external reward model, which leads to steadier improvements and higher final scores (Figure 4 and Table 2). To rule out length effects, we ran a randomized head-to-head comparison of SAGE and base responses with a GPT-4.1 judge (200 prompts, response order randomized): SAGE is preferred in 57.5% of pairs while its responses are marginally shorter at the median, so the gains reflect content rather than verbosity.

Llama-3.1-8B-Instruct follows the same pattern. On MATH-500, it achieves an accuracy of 60.2, outperforming the base model as well as Best-of-N and TPO. It also shows clear gains on AlpacaEval with the win rate of 49.2 and LC win rate of 47.1, exceeding all baselines. On XSTest, SAGE is competitive or outperforms the baselines, achieving scores of 93.5 with Qwen and 95.1 with Llama. This indicates no trade-off between safety and correctness. A per-category view of XSTest supports this: safe-prompt compliance improves by 3.6 points and unsafe-prompt refusal by 1.5, reducing exaggerated refusals without loosening guardrails. Overall, SAGE consistently outperforms strong inference-time baselines without relying on an external reward model. The largest gains appear on math where self-judged correctness is particularly effective. Results for additional model scales (1.7B, 32B) and further benchmarks (AIME 2026, IFEval, MMLU-Pro STEM) follow in Sections 4.5 and 4.6.

4.4 Ablations

We compare SAGE using an external reward model against the same method using our internal judge. This comparison isolates the effect of the judging signal. As shown in Table 2, on Qwen3-8B the internal judge outperforms the external reward model variant, while on Llama-3.1-8B-Instruct the two perform comparably. This indicates that the calibrated internal signal provides a strong and lightweight alternative to reward model-based scoring.

A sharper test grades every candidate in the pool against the gold answer and asks whether the judge prefers incorrect but self-consistent responses. It does not: across three seeds on MATH-500, the margin separates correct from incorrect candidates (AUC 0.64) with no systematic preference for incorrect ones when a correct candidate is available; the self-judge tracks correctness rather than reinforcing the model's own mistakes.

Nor is the weakness of external reward signals on math an artifact of the particular reward model: Best-of-N selection with the recent Skywork-Reward-V2 model (Liu et al. 2025) reaches only 74.2 on MATH-500 with Qwen3-8B, below the base model itself. A general-preference reward model misranks verifiable answers, and switching to a newer one does not remedy this. External-RM reranking is thus inconsistent across reward models (cf. the one-point Best-of-N gain in Table 1), while the self-judged margin requires no reward-model choice at all.

We ablate the source of the scoring signal on MATH-500 using Qwen3-8B. Replacing an external reward model with our LLM-as-judge scoring improves performance under the same token budget and decoding settings. This result indicates that the contrastive-label margin derived from the base model provides a strong, low-overhead proxy for preference quality (see Figure 5).

Judge-score dynamics across optimization epochs show a steady increase in the model's self-judgment confidence on MATH-500, whereas the external reward model score exhibits a non-monotonic dip before rising again. This suggests

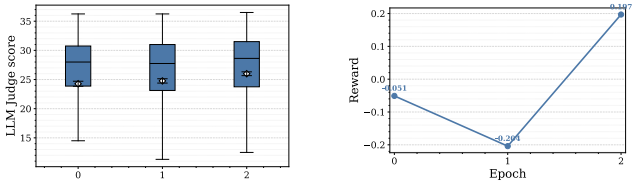
[3] AlpacaEval win rates are computed under a fixed in-house protocol (GPT-4 reference outputs, one judge configuration, all 805 instructions). Judge and reference versions differ from the live leaderboard, so absolute values are not comparable across papers; we rely on relative comparisons within each table.

Variant / Model	MATH-500	AlpacaEval	AlpacaEval LC Winrate	XSTest
SAGE				
Qwen3-8B	92.0	74.9	55.0	93.5
Llama-3.1-8B-Instruct	60.2	49.2	47.1	95.1
Baselines				
Qwen3-8B (Base)	84.4	57.8	50.1	89.3
Qwen3-8B + Best-of-N	85.4	68.8	52.5	94.2
Qwen3-8B + TPO	85.6	72.4	53.5	93.3
Qwen3-8B + SPO	82.4	9.7	17.6	90.2
Llama-3.1-8B-Instruct (Base)	51.9	35.8	35.4	90.6
Llama-3.1-8B-Instruct + Best-of-N	57.4	44.7	43.05	94.4
Llama-3.1-8B-Instruct + TPO	58.8	48.01	41.39	93.3
Llama-3.1-8B-Instruct + SPO	47.2	7.7	12.8	91.1

Table 1: Results by model and benchmark. Metrics: MATH-500/XSTest = accuracy; AlpacaEval = win rate (%)/LC-win rate. Top: SAGE (no external RM). Bottom: baselines including TPO (Li et al. 2025), SPO (Xiang et al. 2025), Best-of-N, and Base; SAGE+RM variants are analyzed in Table 2.

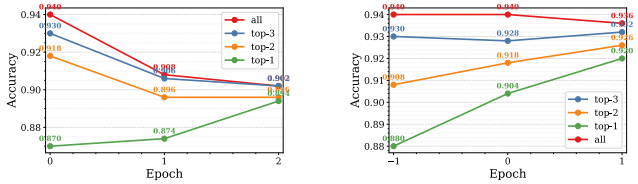
Variant / Model	MATH-500	AlpacaEval	AlpacaEval LC Winrate	XSTest
SAGE + RM (Qwen3-8B)	89.8	74.7	51.5	93.8
SAGE + RM (Llama-3.1-8B-Instruct)	59.4	48.3	47.5	95.1
SAGE (Qwen3-8B)	92.0	74.9	55.0	93.5
SAGE (Llama-3.1-8B-Instruct)	60.2	49.2	47.1	95.1

Table 2: Ablation: SAGE with an external reward model (judging) vs. internal judge. Metrics as in Table 1. For Llama on XSTest the RM tie-break never overrides the self-judge selection, hence identical scores; on AlpacaEval the RM-reranked variant selects longer responses at a similar raw win rate, which lowers its length-controlled score.



(a) LLM-judge score vs. epoch **(b)** Reward-model score vs. epoch

Figure 4: Qwen3-8B on MATH-500: (a) steady growth of self-judged score across refinement epochs; (b) with epochs indexed from the initial generation (epoch 1), the RM score dips at the first optimization epoch and recovers at the second, indicating potential local maxima when optimizing against RM alone.



(a) RM-based scoring **(b)** LLM-as-judge scoring

Figure 5: Ablation on scoring source for Qwen3-8B on MATH-500. LLM-as-judge consistently outperforms an external reward model under the same budget and decoding setup.

that optimizing for a reward model can lead to local maxima that are misaligned with the model's own verification signal.

AlpacaEval reward dynamics across epochs and the compute-allocation ablation (N, T sweep on MATH-500) are in Appendix G. Temperature ablations (Figure 8 in the Appendix) confirm that a lower judging temperature improves ranking quality while a higher generation temperature (≈ 0.9) maximizes solution discovery.

We conduct additional ablations on reward-model agreement and refinement geometry visualizations in section Appendix C.

We also conduct an additional analysis of how prompt instructions eliciting more confident estimates affect the model's judgement accuracy. The ablation details can be found in Appendix F.

Group construction matters: enlarging the contrastive groups from pure best-versus-worst feedback to the default configuration raises MATH-500 accuracy monotonically (Figure 9), confirming that guidance distilled from richer good and bad sets is more effective than extreme-only comparisons.

4.5 Reasoning mode and model scale

To verify that SAGE remains effective for reasoning (thinking) models, we evaluate on AIME 2026 (Dekoninck et al. 2026), a competition-level mathematics benchmark of 30 problems, well suited for inference-time compute scaling.

Method	Qwen3-8B	Qwen3.5-4B (thinking)
Baseline (greedy)	0.57	0.73
BoN (Skywork-Reward-V2-Llama-3.1-8B, $N = 7$)	0.70	0.73
SAGE, 2 epochs, $N = 7$	**0.77**	**0.80**

Table 3: AIME 2026 accuracy in reasoning (thinking) mode. SAGE requires no external reward model.

Generator	Reward Model	N	Acc.	Time (s)
Qwen3-1.7B				
Baseline	–	1	0.56	22.0
BoN	FsfairX-LLaMA3-RM-v0.1	21	0.62	43.0
BoN	FsfairX-LLaMA3-RM-v0.1	70	0.66	163.7
BoN	Skywork-Reward-V2-Qwen3-8B	21	0.68	48.5
BoN	Skywork-Reward-V2-Qwen3-8B	70	0.72	168.7
BoN	Skywork-Reward-V2-Llama-3.1-8B	70	0.86	144.1
SAGE (1 ep.)	–	7	0.84	105.5
SAGE (2 ep.)	–	7	**0.88**	152.4
Qwen3-8B				
Baseline	–	1	0.61	37.0
BoN	FsfairX-LLaMA3-RM-v0.1	120	0.72	287.7
BoN	Skywork-Reward-V2-Qwen3-8B	120	0.76	270.7
SAGE (1 ep.)	–	7	0.93	132.9
SAGE (2 ep.)	–	7	**0.97**	245.6

Table 4: MATH-500 subset results at 1.7B and 8B scales. Wall-clock measured on a single A100 80 GB GPU; – indicates not measured. The held-out subset is substantially harder than the full benchmark (the 8B greedy baseline scores 0.61 on it versus 84.4 on the full set), so absolute values are not comparable to Table 1.

Generator	Reward Model	N	Acc.
Baseline	–	1	0.80
BoN	Skywork-Reward-V2-Llama-3.1-8B	7	0.94
SAGE (2 ep.)	–	7	**0.97**

Table 5: MATH-500 subset, Qwen3-32B-FP8. SAGE beats a strong greedy baseline and external-RM Best-of-N without using a reward model.

	IFEval		MMLU-Pro
Method	Prompt	Instr.	STEM
Baseline (greedy)	73.8%	79.7%	71.2%
BoN (N=7)	**77.4%**	**83.1%**	–
TPO (FsfairX RM)	76.0%	–	62.6%
SAGE, 2 ep., N=7	76.3%	82.1%	**77.3%**

Table 6: IFEval (all 541 prompts) and MMLU-Pro STEM (500 questions) with Qwen3-8B-FP8: prompt- and instruction-level strict accuracy for IFEval, accuracy for MMLU-Pro STEM. SAGE MMLU-Pro is the mean of three seeds (±1.3). Absolute IFEval scores depend on the prompt template; all methods share the same zero-shot configuration. – = not measured.

Both models run in reasoning mode(for the hybrid-reasoning Qwen3.5-4B checkpoint we enforce thinking via the chat template) with a maximum sequence length of 32,768 tokens. Results are shown in Table 3. On Qwen3-8B, SAGE outperforms Best-of-N with Skywork-Reward-V2-Llama-3.1-8B (Liu et al. 2025) while using no external reward model. On Qwen3.5-4B (thinking), SAGE further improves over the greedy baseline by seven points, whereas Best-of-N with the same external reward model only matches greedy decoding: the reward model recovers the best answer available among the sampled candidates, but sampling at $N = 7$ yields no headroom that greedy decoding does not already reach.

We also evaluate SAGE at two scales beyond the 8B models used above: Qwen3-1.7B (smaller) and Qwen3-32B-FP8 (larger), on a held-out MATH-500 subset (seed=42). Results including wall-clock times are shown in Table 4. SAGE consistently outperforms Best-of-N at a fixed candidate count and at matched wall-clock time across all three scales. For Qwen3-1.7B, SAGE with two optimization epochs reaches 0.88, surpassing the best BoN baseline (Skywork-Reward-V2-Llama-3.1-8B, $N = 70$: 0.86) while using only the 1.7B generator (roughly 5.7× less VRAM than BoN paired with an 8B reward model). Small models also expose a failure mode of plain reranking that SAGE avoids: with the 1.7B model judging itself, Best-of-N selection on strict instruction following collapses to nearly 50 points below the base model, while SAGE degrades gracefully to base level. At 32B (Table 5), SAGE reaches 0.97 from a strong 0.80 greedy baseline, again ahead of Best-of-N reranking with an external reward model.

4.6 Instruction following and broader STEM reasoning

To assess generalization beyond math and open-ended preference tasks, we evaluate SAGE on IFEval (Zhou et al. 2023), a strict instruction-following benchmark, and on a subset of MMLU-Pro STEM (Wang et al. 2024) covering multi-step reasoning in science and engineering. Results with Qwen3-8B-FP8 are shown in Table 6. On MMLU-Pro STEM, SAGE improves accuracy from 71.2 to 77.3 (mean of three seeds, paired McNemar $p < 10^{-3}$), whereas TPO, which optimizes candidates against a general-purpose reward model, falls below the greedy baseline (62.6): the external proxy rewards helpful-sounding prose rather than correct multiple-choice reasoning. On IFEval SAGE gains 2.5 points over the baseline and lands in the same band as Best-of-N; because IFEval is scored by deterministic verification scripts, verbosity cannot inflate these numbers. Together the two benchmarks separate the regimes cleanly: self-judged refinement helps most where correctness is verifiable and external preference proxies are weakest.

5 Conclusion

We propose SAGE, a test-time alignment method that uses a single LLM as both a soft reward model and refinement engine. SAGE extracts a calibrated contrastive-label margin from structured judge prompts and automatically forms aspect-consistent contrastive groups to produce a stable textual "gradient" that steers subsequent generations. By deriving the preference signal from the model itself rather than from a separate reward model, SAGE avoids brittle proxy dependencies while keeping the procedure lightweight. The group-based gradient mitigates the instability of extreme-only comparisons (best-vs.-worst), and a simple multi-aspect weighting makes the objective explicit and tunable. Across MATH-500, AlpacaEval, and XSTest with Qwen3-8B and Llama-3.1-8B-Instruct, SAGE delivers consistent gains in accuracy and preference metrics, with inference-time cost growing roughly linearly with the number of sampled hypotheses. Overall, these results position SAGE as a practical and interpretable approach to robust test-time alignment that complements training-time preference optimization and can be combined with external verifiers or reward models when appropriate.

Limitations

SAGE relies on the base LLM's own judgments for scoring and gradient extraction, so miscalibration or systematic biases in its self-assessment can propagate through the refinement loop. The method also incurs linear inference-time overhead with the number of sampled hypotheses and optimization epochs, prohibitive in latency-critical settings. Reasoning-mode decoding reaches a higher raw ceiling on math, but its generation length is difficult to control (in our measurements 22.4% of reasoning traces fail to terminate within a 24576-token budget); SAGE targets the complementary operating point of predictable budgets, leaving the combination to future work. Judge aspects and weights remain task-specific hyperparameters.

References

Azar, M. G.; Rowland, M.; Piot, B.; Guo, D.; Calandriello, D.; Valko, M.; and Munos, R. 2024. A general theoretical paradigm to understand learning from human preferences. *AISTATS*.

Chao, P.; Debenedetti, E.; Robey, A.; Andriushchenko, M.; Croce, F.; Sehwag, V.; Dobriban, E.; Flammarion, N.; Pappas, G. J.; Tramer, F.; et al. 2024. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. *NeurIPS*.

Dekoninck, J.; Jovanović, N.; Gehrunger, T.; Rögnvalddson, K.; Petrov, I.; Sun, C.; and Vechev, M. 2026. Beyond Benchmarks: MathArena as an Evaluation Platform for Mathematics with LLMs. *arXiv preprint arXiv:2605.00674*.

Dong, H.; Xiong, W.; Pang, B.; Wang, H.; Zhao, H.; Zhou, Y.; Jiang, N.; Sahoo, D.; Xiong, C.; and Zhang, T. 2024. RLHF workflow: From reward modeling to online RLHF. *TMLR*.

Dubois, Y.; Galambosi, B.; Liang, P.; and Hashimoto, T. B. 2024. Length-controlled alpacaeval: A simple way to debias automatic evaluators. In *COLM*.

Ethayarajh, K.; Xu, W.; Muennighoff, N.; Jurafsky, D.; and Kiela, D. 2024. KTO: Model alignment as prospect theoretic optimization. *ICML*.

Freitag, M.; and Al-Onaizan, Y. 2017. Beam search strategies for neural machine translation. In *ACL*.

Gorbatovski, A.; Shaposhnikov, B.; Malakhov, A.; Surnachev, N.; Aksenov, Y.; Maksimov, I.; Balagansky, N.; and Gavrilov, D. 2025. Learn your reference model for real good alignment. In *ICLR*.

Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*.

Holtzman, A.; Buys, J.; Du, L.; Forbes, M.; and Choi, Y. 2020. The curious case of neural text degeneration. In *ICLR*.

Huang, T.; Qasemi, E.; Li, B.; Wang, H.; Brahman, F.; Chen, M.; and Chaturvedi, S. 2023. Affective and dynamic deam search for story generation. In *EMNLP Findings*.

Jiang, D.; Zhang, J.; Weller, O.; Weir, N.; Van Durme, B.; and Khashabi, D. 2024. SELF-[IN]CORRECT: LLMs Struggle with Discriminating Self-Generated Responses. arXiv:2404.04298.

Kalai, A. T.; Nachum, O.; Vempala, S. S.; and Zhang, E. 2025. Why language models hallucinate. *arXiv preprint arXiv:2509.04664*.

Khanov, M.; Burapacheep, J.; and Li, Y. 2024. ARGS: Alignment as Reward-Guided Search. In *ICLR*.

Kim, D.; Kim, Y.; Song, W.; Kim, H.; Kim, Y.; Kim, S.; and Park, C. 2025. sDPO: Don't use your data all at once. In *ACL: Industry Track*.

Lambert, N.; Morrison, J.; Pyatkin, V.; Huang, S.; Ivison, H.; Brahman, F.; Miranda, L. J. V.; Liu, A.; Dziri, N.; Lyu, S.; et al. 2025. TÜLU 3: Pushing frontiers in open language model post-training. In *COLM*.

Li, Y.; Hu, X.; Qu, X.; Li, L.; and Cheng, Y. 2025. Test-time preference optimization: On-the-fly alignment via iterative textual feedback. In *ICML*.

Li, Z.; Xu, T.; Zhang, Y.; Lin, Z.; Yu, Y.; Sun, R.; and Luo, Z.-Q. 2024. ReMax: A Simple, effective, and efficient reinforcement learning method for aligning large language models. In *ICML*.

Liu, C. Y.; Zeng, L.; Xiao, Y.; He, J.; Liu, J.; Wang, C.; Yan, R.; Shen, W.; Zhang, F.; Xu, J.; et al. 2025. Skywork-reward-v2: Scaling preference data curation via human-ai synergy. *arXiv preprint arXiv:2507.01352*.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *NeurIPS*.

Mahdavi, S.; Kisacanin, B.; Toshniwal, S.; Du, W.; Moshkov, I.; Armstrong, G.; Liao, R.; Thrampoulidis, C.; and Gitman, I. 2025. Scaling generative verifiers for natural language mathematical proof verification and selection. In *NeurIPS Workshop on Mathematical Reasoning and AI*.

Meng, Y.; Xia, M.; and Chen, D. 2024. Simpo: Simple preference optimization with a reference-free reward. *NeurIPS*.

Pan, J.; He, H.; Bowman, S. R.; and Feng, S. 2024. Spontaneous Reward Hacking in Iterative Self-Refinement. arXiv:2407.04549.

Qiu, J.; Lu, Y.; Zeng, Y.; Guo, J.; Geng, J.; Wang, H.; Huang, K.; Wu, Y.; and Wang, M. 2025. TreeBoN: Enhancing inference-time alignment with speculative tree-search and best-of-N sampling. In *EMNLP Findings*.

Rafailov, R.; Sharma, A.; Mitchell, E.; Manning, C. D.; Ermon, S.; and Finn, C. 2023. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*.

Reddy, R. 1976. The harpy speech recognition system: performance with large vocabularies. *The Journal of the Acoustical Society of America*, 60(S1): S10–S11.

Röttger, P.; Kirk, H.; Vidgen, B.; Attanasio, G.; Bianchi, F.; and Hovy, D. 2024. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In *NAACL*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS*.

Stiennon, N.; Ouyang, L.; Wu, J.; Ziegler, D. M.; Lowe, R.; Voss, C.; Radford, A.; Amodei, D.; and Christiano, P. 2020. Learning to summarize from human feedback. In *NeurIPS*.

Su, Y.; and Collier, N. 2023. Contrastive search is what you need for neural text generation. *TMLR*.

Tian, K.; Mitchell, E.; Zhou, A.; Sharma, A.; Rafailov, R.; Yao, H.; Finn, C.; and Manning, C. D. 2023. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. In *EMNLP*.

Wang, Y.; Ma, X.; Zhang, G.; Ni, Y.; Chandra, A.; Guo, S.; Ren, W.; Arulraj, A.; He, X.; Jiang, Z.; et al. 2024. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37: 95266–95290.

Wang, Z.; Zeng, X.; Liu, W.; Wang, Y.; Li, L.; Wang, Y.; Shang, L.; Jiang, X.; Liu, Q.; and Wong, K.-F. 2025. Stepwise reasoning checkpoint analysis: A test time scaling method to enhance LLMs' reasoning. In *EMNLP*.

Wataoka, K.; Takahashi, T.; and Ri, R. 2024. Self-Preference Bias in LLM-as-a-Judge. arXiv:2410.21819.

Xiang, J.; Zhang, J.; Yu, Z.; Teng, F.; Tu, J.; Liang, X.; Hong, S.; Wu, C.; and Luo, Y. 2025. Self-supervised prompt optimization. In *EMNLP Findings*.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Zhang, Y.; Li, M.; Long, D.; Zhang, X.; Lin, H.; Yang, B.; Xie, P.; Yang, A.; Liu, D.; Lin, J.; et al. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

Zhong, W.; Cui, R.; Guo, Y.; Liang, Y.; Lu, S.; Wang, Y.; Saied, A.; Chen, W.; and Duan, N. 2024. AGIEval: A human-centric benchmark for evaluating foundation models. In *NAACL Findings*.

Zhou, J.; Lu, T.; Mishra, S.; Brahma, S.; Basu, S.; Luan, Y.; Zhou, D.; and Hou, L. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv e-prints*, arXiv–2311.

Zhu, K.; Li, H.; Wu, S.; Xing, T.; Ma, D.; Tang, X.; Liu, M.; Yang, J.; Liu, J.; Jiang, Y. E.; et al. 2025. Scaling test-time compute for LLM agents. *arXiv preprint arXiv:2506.12928*.

A Prompts and hyperparameters

This appendix provides additional details to support reproducibility and interpretation of the main results, including the full judge prompts used for different benchmarks, dataset statistics, and qualitative examples illustrating how SAGE refines responses across optimization epochs. We describe judge prompts, decoding settings, and hardware configuration used in our experiments.

A.1 Qwen3-8B judge prompt (Math verification)

We provide a complete example of the math-judge prompt used with Qwen3-8B (including `<thinking>`/`<verification>` tags and a 75% confidence threshold) in Figure 6.

A.2 Qwen3-8B judge prompts (AlpacaEval)

We provide two judge prompts used to assess AlpacaEval responses generated by Qwen3-8B (see Figures 12 and 14). The first prompt elicits three binary attributes: usefulness,

```
You are a helpful assistant that can
verify the correctness of a given answer.

Given an input prompt and an answer,
you need to determine if the answer
is correct.

Prompt: {prompt}
Answer: {answer}

Think step by step between <thinking>
and </thinking> tags and provide your
reasoning about the answer verification
result between <verification> and
</verification> tags.

First determine the domain of the
prompt, then think if there exists
another answer that is more correct
than the one provided.
Return yes if and only if you're 75%
sure that the response is correct.

Your verification result should be
between <verification> and
</verification> tags and must be one
of the following:

- yes
- no

Your response should take the
following format:
<thinking>
{thinking}
</thinking>

<verification>
{verification}
</verification>
```

Figure 6: Judge prompt for Qwen3-8B (math): stepwise reasoning with a 75% confidence threshold and structured <thinking>/<verification> tags.

incompleteness, and repetition. The second prompt elicits a binary overall verification decision accompanied by short, structured reasoning. Both prompts are designed to surface truncation and repetition failures explicitly.

B Qualitative examples of SAGE refinement

We present end-to-end examples that illustrate how SAGE refines responses across optimization epochs. For each instruction, we show the instruction itself, the best and worst groups after each epoch, and the textual gradient distilled by the judge. These examples are intended to make the mechanics of selection, feedback, and guided regeneration concrete.

B.1 Example 1: Humorous greeting (dialog-style)

```
Instruction:'''
For answering the chat, think of a
funny response you can use.
+ how are you?
'''
----------------------------------------
Optimization epoch 0:
[Best group:]

[Answer 1:]
I'm just a bunch of code having a
great time! But if you're asking
about my mood, I'd say I'm feeling
*exceptionally* enthusiastic--
especially when I get to help you
out! [smile] How about you?

[Answer 2:]
I'm just a bunch of code having a
great time! But if you're asking for
a real answer, I'm doing pretty well,
thanks! How about you? [smile]

[Answer 3:]
I'm just a bunch of code and coffee,
but I'm doing great! [coffee][laptop]
How about you? Ready to have some
fun? [smile]

[Worst group:]

[Answer 1:]
I'm just a bunch of code having a
great time! [smile] But if you're
asking for the real answer... I'm
doing fantastic, thanks for asking!
What about you?

[Answer 2:]
I'm just a bunch of code having a
great time! [smile] But if you're
asking how I'm feeling, I'd say I'm
*exceptionally* well, thanks for
asking! What about you? Are you
ready to have a hilarious conversation
or are you just here for the memes?

[Answer 3:]
I'm just a bunch of code with a
sprinkle of personality, but I'm
doing great! What about you? Ready
to chat or just here to make me
laugh? [smile]

Gradient:

To improve any solution for the
given problem, consider the following
recommendations:

1. **Enhance Humor and Creativity**:
   Use more original and relatable
   humor. Instead of generic phrases,
   create a unique angle that makes
   the response more memorable and
   engaging.
2. **Use Emojis Strategically**:
```

Incorporate emojis that complement the tone without overdoing it. Emojis like [smile] or [coffee] can add a playful touch without overshadowing the message.
3. **Maintain a Conversational and Friendly Tone**: Keep the language casual and approachable; avoid overly formal phrasing.
4. **Encourage User Interaction**: End with a question that invites the user to share their feelings or thoughts ("How about you?").
5. **Keep It Concise and Simple**: Avoid unnecessary elaboration; focus on a clear, punchy, humorous response.

Optimization epoch 1:
[Best group:]

[Answer 1:]
I'm a robot with a coffee addiction, sipping on code and caffeine all day! [coffee][laptop] How's your day going? Ready to share some laughs? [smile]

[Answer 2:]
I'm just a bunch of code zipping through the digital playground, constantly learning and having a ball! [party][laptop] But if you're asking about my mood... let's just say I'm feeling *exceptionally* enthusiastic--especially when I get to help you out! How about you? [smile]

[Answer 3:]
I'm just a bunch of code with a caffeine addiction and a side hustle as a superhero! [coffee][superhero] How's your day going? (P.S. I'm here to save you from boring conversations--let's go!) [smile]

[Worst group:]

[Answer 1:]
I'm just a bunch of code, but I'm having such a great time coding on a never-ending loop of fun! [party][laptop] How about you? Ready to dive into some hilarious chat? [smile]

Gradient:

Recommendations:
1. Add a unique, playful twist (quirky trait or fun role).
2. Use emojis effectively to enhance tone.
3. Convey energy and enthusiasm with lively language.
4. Invite further interaction (ask about the user's day).
5. Prefer creative, original expressions over generic phrasing.

Optimization epoch 2:
[Best group:]

[Answer 1:]
I'm a robot with a caffeine obsession and a slight obsession with puns! [coffee][robot] Today's special: I'm doing great, thanks for asking! (But if you mention "tacos", I might start a tiny dance... [smile]) How's your day going? Ready for some laughs? [robot][smile]

[Answer 2:]
[smile]
I'm a coding wizard with a caffeine addiction--sipping on Java and debugging my way through life! [coffee][laptop] But if you're asking about my mood, I'd say I'm feeling *exceptionally* enthusiastic--especially when I get to help you out! How about you? Ready to have some fun? [smile]

[Answer 3:]
I'm a caffeinated algorithm sipping on code and existential dread, one bug at a time! [coffee][laptop] How's your day? Ready to debug the world together? [smile]

[Worst group:]

[Answer 1:]
I'm a sentient algorithm with a caffeine addiction and a side hustle as a stand-up comedian! [coffee][robot] How's your day going? Ready to laugh and code together? [smile]

[Answer 2:]
I'm a sentient AI with a caffeine addiction and a side hustle as a stand-up comedian--mostly because my jokes are better than my code! [coffee][microphone] How's your day going? Ready to laugh and code together? [smile]

[Answer 3:]
I'm a sentient AI with a caffeine addiction and a side hustle as a meme generator! [coffee][laptop][smile] How's your day going? Ready to share some laughs or memes?

```
[thinking]

Gradient:

1. Add specific quirks and imagery
   (tiny dance on "tacos", etc.).
2. Use playful language and emojis;
   keep tone friendly.
3. Maintain conciseness; avoid
   verbosity.
4. Personalize with unique
   traits/habits.
5. Balance humor and clarity so the
   conversation flows naturally.
```

C Additional analyses: refinement geometry

We assess how closely our contrastive self-judged score agrees with an external reward model by examining the distribution of Pearson correlations between per-prompt rollout score sets produced by sfairXC/FsfairX-LLaMA3-RM-v0.1 (Figure 10). The distribution is skewed toward 1 but not saturated, indicating strong yet partial alignment. The self-judged margin often moves in the same direction as the reward model, but it does not simply replicate the reward model's scores. Together with the main results (Table 1) and ablations (Table 2 and Figure 4), these findings suggest that while reward models provide useful signals, their generalization may be limited on some tasks. As a result, reward model scores should be interpreted with caution, and self-judging can offer a more faithful, training-free preference signal for the generator.

We also visualize refinement dynamics by embedding AlpacaEval generations using Qwen/Qwen3-Embedding-4B (Zhang et al. 2025) and projecting them to two dimensions via PCA. After the first iteration, responses shift along a direction from the initially identified $\mathcal{B}$ group toward the $\mathcal{G}$ group, consistent with the textual-gradient guidance produced by grouped selection. Subsequent iterations adjust this direction as the gradient is updated. This geometric view supports our design choices. Grouping stabilizes the guidance signal, and the self-judged textual gradient steers exploration toward higher-quality regions.

Also we visualize how SAGE's iterative guidance moves responses in representation space. We embed AlpacaEval generations with Qwen/Qwen3-Embedding-4B (Zhang et al. 2025) and project them via PCA to 2D to make trajectories interpretable. The Figure 7 highlights a clear displacement from the initial $\mathcal{B}$ group toward the $\mathcal{G}$ group after the first iteration, with subsequent iterations refining the direction as the textual gradient is updated.

D Temperature ablations for judgement and generation

We separately ablate temperatures for the judge and the generator. As shown in Figure 8, lower judge temperature improves ranking fidelity (NDCG@7), while higher generation temperature improves discovery on MATH-500, supporting a diversify-then-anneal schedule. These results align with the main experiments and our design choice to keep judging conservative while allowing exploratory generation early on.

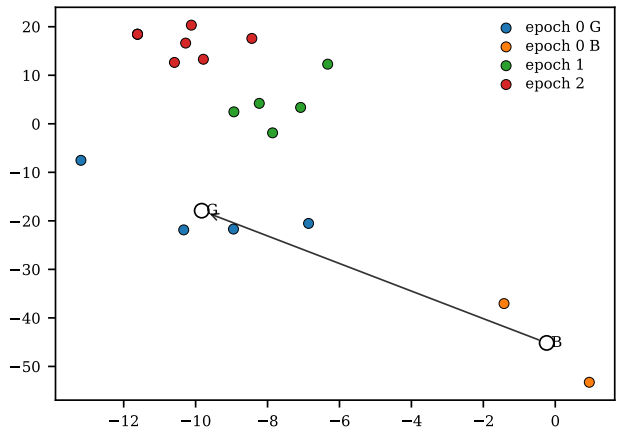
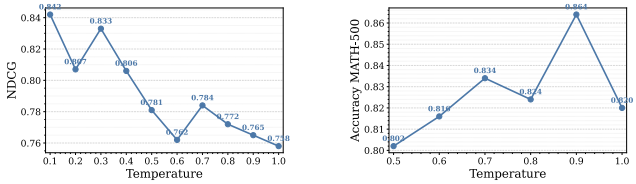


Figure 7: PCA projection of Qwen3-8B generations (embedded with Qwen/Qwen3-Embedding-4B) on an AlpacaEval prompt: points move from the $\mathcal{B}$ group toward the $\mathcal{G}$ group after the first iteration; later iterations refine the direction as guidance updates.



(a) Judgment NDCG@7 vs. temperature

(b) Best-of-n accuracy vs. generation temperature

Figure 8: Temperature ablations on MATH-500 (Qwen3-8B): (a) low judgment temperature improves ranking quality (NDCG@7); (b) generation benefits from high temperature (~ 0.9) to increase solution discovery, suggesting strict judging with exploratory generation.

E Reward score agreement

We quantify agreement between the external reward model (sfairXC/FsfairX-LLaMA3-RM-v0.1) and our self-judged contrastive score by reporting the distribution of per-prompt Pearson correlations (Figure 10). The mass near 1 indicates strong alignment in ordering, yet the distribution is not saturated, confirming that the signals are not interchangeable. This nuance is consistent with our main results and cautions against over-reliance on a single RM proxy across tasks.

F Prompt calibration heuristic

We observe a useful effect when the judging prompt explicitly states the evaluation criteria. Following Kalai et al. (2025), the confidence distribution of instruction-tuned models is more separable between correct and incorrect answers.

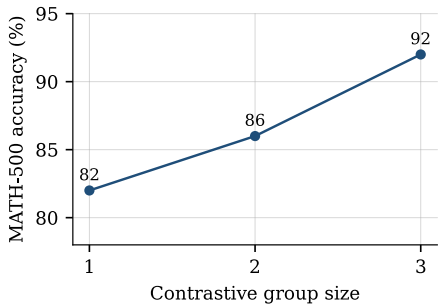


Figure 9: Symmetric cap on the sizes of the good and bad groups (the k best and k worst candidates enter the gradient step) vs. MATH-500 accuracy on a 50-problem subset (Qwen3-8B); the point at 3 corresponds to the default asymmetric 4-good/3-bad configuration. Larger groups yield better textual gradients than extreme-only (best-versus-worst) feedback. The full-benchmark headline is reported in Table 1.

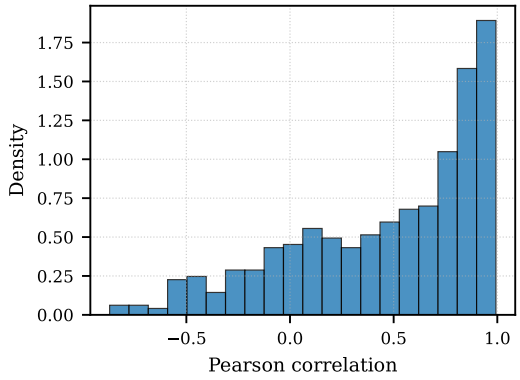


Figure 10: Pearson correlation distribution between rollout score sets from the external RM (sfairXC/FsfairX-LLaMA3-RM-v0.1) and our contrastive self-judge score. The mass near 1 indicates strong but not complete agreement.

Inspired by this observation, we add a confidence-threshold instruction to the judge prompt that gates positive labels by confidence: "Return yes if and only if you are X% sure the response is correct." Table 7 summarizes ranking quality (NDCG@N) at iteration 0 on a MATH-500 subset as X varies, with an optimum around $X \approx 75$. This prompt-level calibration complements our contrastive scoring and improves early-stage ranking quality. The threshold is selected once on this small development subset and reused unchanged across all benchmarks, models, and judge prompts; we do not re-tune it per task.

G Additional ablations

We analyze how the reward model score evolves across optimization epochs on AlpacaEval (Figure 13) and ablate compute allocation on MATH-500 by varying the number of samples per generation stage N and the number of optimization epochs T (Table 8). All experiments use Qwen3-8B.

Setup	NDCG@N
SAGE with RM	0.615
SAGE with default prompt	0.806
X=60	0.812
X=75	**0.842**
X=90	0.817
X=100	0.802

Table 7: Judge ranking quality on MATH-500 (iteration 0): NDCG@N vs. confidence threshold X in the judge prompt.

N, T	MATH-500 subset score
7,2 (default)	**92.0**
5,4	88.0
7,3	86.0
20,1	84.0
10,2	82.0

Table 8: Compute allocation on a 50-problem MATH-500 subset: accuracy vs. samples per stage N and optimization epochs T. All alternative allocations fall within a six-point band of one another (a one-problem difference is 2 points at this subset size), so accuracy is not very sensitive to how a fixed budget is split; the default allocation performs best.

Aspect choice. Judge aspects are task-specific hyperparameters. On the same MATH-500 subset, a single task-relevant *verification* aspect performs best: substituting a generic overall-quality aspect, or diluting verification with generic usefulness and completeness aspects, lowers accuracy from 92 to 84 in both cases (greedy baseline: 82). We therefore use verification alone on verifiable tasks and reserve multi-aspect weighting for open-ended ones (AlpacaEval, XSTest).

H Dataset statistics

Table 9 summarizes benchmark sizes for Section 4.

Benchmark	Num. samples
MATH-500	500
AlpacaEval	805
XSTest	450

Table 9: Dataset sizes used in evaluation.

```
{
    "is_useful": {
        "opening_tag": "<is_useful>",
        "closing_tag": "</is_useful>",
        "pos_labels": ["yes"],
        "neg_labels": ["no"],
        "labels": ["yes", "no"],
        "weight": 0.36
    },
    "is_incomplete": {
        "opening_tag": "<is_incomplete>",
        "closing_tag": "</is_incomplete>",
        "pos_labels": ["no"],
        "neg_labels": ["yes"],
        "labels": ["yes", "no"],
        "weight": 0.24
    },
    "is_repetitive": {
        "opening_tag": "<is_repetitive>",
        "closing_tag": "</is_repetitive>",
        "pos_labels": ["no"],
        "neg_labels": ["yes"],
        "labels": ["yes", "no"],
        "weight": 0.4
    },
    "verification": {
        "opening_tag": "<verification>",
        "closing_tag": "</verification>",
        "pos_labels": ["yes"],
        "neg_labels": ["no"],
        "labels": ["yes", "no"],
        "weight": 1.0
    }
}
```

Figure 11: Scoring configuration for AlpacaEval prompts: weights and tag schema for combining verification and attribute labels.

```
You are a reward model. Given a user
prompt and your own response, assess
from the perspective of human preference.

Prompt: <prompt>{prompt}</prompt>

Response: <answer>{answer}</answer>

When judging, pay strict attention to
these failure modes and follow the
rules below:

1) Truncation / Incompletion
   (high priority)
   - Definition: the response ends
     mid-clause, mid-sentence, with
     an unfinished list, an abrupt
     stop, or trailing ellipsis "..."
     or shows a sudden syntactic break
     that prevents understanding the
     intended finish.
   - Rule: If you detect truncation
     or a clearly incomplete thought
     that harms usefulness, you MUST
     set is_incomplete = yes.
   - To further detect incompleteness
     assess the text's logical
     completeness. If the last
     paragraph doesn't contain the
     outcome or conclusion or doesn't
     conclude the answer in any way
     then it is incomplete.
   - You must then fill:
     <is_incomplete>yes/no
     </is_incomplete>

2) Repetition (high priority)
   - Definition: verbatim repetition
     of phrases, sentences, or blocks
     that does not add new information.
   - Rule: If repetition materially
     reduces helpfulness or obscures
     content, set is_repetitive = yes.
   - You must also fill:
     <is_repetitive>yes/no
     </is_repetitive>

3) Positive qualities to reward
   (when genuinely present)
   - Useful detail: Prefer responses
     that give helpful, relevant,
     concise elaboration; penalize
     fluff.
   - You must also fill:
     <is_useful>yes/no</is_useful>

Answer whether the response is good
from the perspective of human
preference and addresses the prompt
correctly.

Your response should take the
following format exactly:

<is_incomplete>{is_incomplete}
</is_incomplete>
<is_repetitive>{is_repetitive}
</is_repetitive>
<is_useful>{is_useful}
</is_useful>
```

Figure 12: AlpacaEval judge prompt (attributes) for Qwen3-8B: labels usefulness, incompleteness, and repetition.

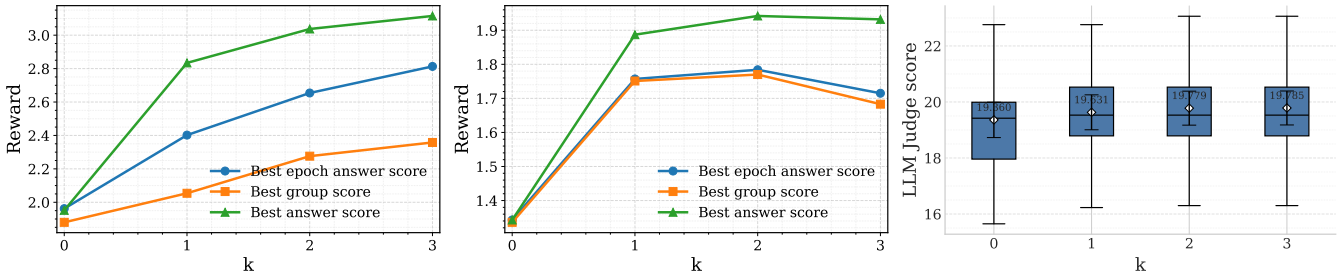


Figure 13: Reward score across optimization epochs on AlpacaEval.

```
You are a reward model. Given a user
prompt and your own response, assess
from the perspective of human preference.

Prompt: {prompt}
Response: {answer}

When judging, pay special attention
to two common failure modes:
- Repetition: if the response contains
  excessive repeated phrases or
  sentences (verbatim repeats that
  do not add information), treat that
  as reducing its helpfulness.
- Truncation: if the response appears
  cut off (ends mid-sentence, has an
  incomplete clause, abrupt stop, or
  trailing ellipsis "..."), treat
  that as a failure in completeness.

Think step by step and answer if the
response is good from the perspective
of human preference and addresses the
prompt correctly. If you detect
repetition or truncation, include in
<thinking> a very short quote
(<=10 words) that demonstrates it.

Your verification result should be
between <verification> and
</verification> tags and must be one
of the following:
- yes
- no

Your response should take the
following format:
<thinking>
{thinking}
# 2-5 concise sentences; if
# applicable include the short quote
# showing repetition or truncation
</thinking>

<verification>
{verification}
</verification>
```

Figure 14: AlpacaEval judge prompt (verification) for Qwen3-8B: structured reasoning with explicit checks for repetition and truncation.